START DATA MANAGEMENT

TRATAMENTO DE EXCEÇÕES COM PYTHON

4

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE FIGURAS

Figura 1 - Exceção 6
Figura 2 – Esquema geral da exceção..... 7
Figura 3 – Esquema lançamento / propagação da exceção 12

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Estrutura básica do try/except 6
Código-fonte 2 – Pedindo a idade do aluno com try/except..... 7
Código-fonte 3 – Pedindo a idade do aluno com try/except/else 8
Código-fonte 4 – Pedindo a idade do aluno com try/except/else/finally..... 8
Código-fonte 5 – Exemplo com except sem especificar o tipo de exceção 10
Código-fonte 6 – Exemplo de raise com especificação da mensagem de erro 13
Código-fonte 7 – Exemplo de raise com especificação da mensagem de erro e propagação 13
Código-fonte 8 – Exemplo de raise com especificação da mensagem de erro e propagação 15
Código-fonte 9 – Média de valor gasto por item em uma compra 17
Código-fonte 10 – Média de valor gasto por item em uma compra com erro personalizado 18

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

SUMÁRIO

1 EXCEÇÕES 5
1.1 Quando a gasolina acaba 5
1.2 Vamos tentar? Finalmente…..... 6
1.3 O que Python conhece sobre você? 9
1.4 E quando eu quero lançar ou propagar uma exceção? 11
1.5 Exceções Personalizadas 14
1.5.1 Classe base Exception 14
2 HANDS ON: HORA DA PRÁTICA 16
2.1 Com ou sem exceção..... 16
2.1.1 Quanto estamos gastando mesmo? 16
2.1.2 Nada de ficar devendo! 17
REFERÊNCIAS..... 19

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1 EXCEÇÕES

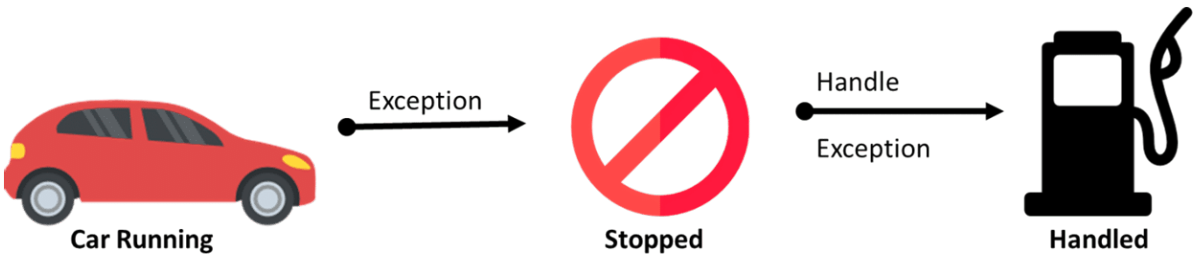
Em programação, uma exceção é um evento anômalo que ocorre durante a execução de um programa e que afeta o seu comportamento normal. Exceções são utilizadas para lidar com situações inesperadas, tais como erros de entrada/saída, falta de memória, tentativas de acesso a recursos não autorizados, entre outros. Quando uma exceção é lançada, ela interrompe a execução normal do programa e é propagada até ser tratada por um trecho de código que é especificamente escrito para lidar com ela.

O objetivo principal de usar exceções é separar o código que detecta e lida com situações anômalas do código que executa a lógica principal do programa. Isso torna o código mais claro e fácil de manter, uma vez que o tratamento de exceções pode ser concentrado em um único local em vez de estar espalhado por todo o código. Além disso, o uso de exceções permite que as situações de erro sejam tratadas de forma uniforme, independentemente de onde elas ocorreram no código, tornando a depuração mais fácil.

1.1 Quando a gasolina acaba

Imagine que você está dirigindo um carro da programação, tudo está correndo bem e de repente, BUM! Seu motor para por falta de gasolina.

Isso é como uma exceção no mundo da programação. A execução do seu programa está correndo bem, mas de repente algum evento inesperado acontece e interrompe a jornada. Assim como você precisa abastecer o carro para continuar a viagem, você precisa lidar com a exceção para que o programa possa continuar sua execução.



PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Figura 1 - Exceção
Fonte: Google Imagens (2016)

Mas não se preocupe, como bom motorista de programação, você tem um plano de emergência para lidar com essas situações - você pode simplesmente colocar o seu código de tratamento de exceções para resolver o problema e seguir em frente!

1.2 Vamos tentar? Finalmente…

Imagine que você está programando e o trecho de código que você está escrevendo pede ao usuário que ele entre com a sua idade. Mas durante a execução do programa o usuário erra e ao invés de fornecer a idade ele fornece o próprio nome. Isso causará uma exceção no seu programa, pois é uma situação inesperada.

Quando uma exceção é lançada, significa que uma situação inesperada ocorreu durante a execução do programa, e para lidar com essa situação, a exceção é "lançada". Isso interrompe a execução normal do programa e fornece a oportunidade para que o programador trate a exceção.

A estrutura de controle "try / except" em Python permite capturar e tratar exceções de forma clara e concisa e no Python não são necessários blocos específicos para exceções, como uma função específica, ou seja, "try / except" pode ser escrito em qualquer área do seu código. A sintaxe básica do bloco "try / except" é a seguinte:

```
try:
   # código que pode lançar exceção
except ExceptionType:
   # código para tratar a exceção lançada
```

Código-fonte 1 – Estrutura básica do try/except
Fonte: Elaborado pelo autor (2022)

No nosso exemplo teríamos:

```
try:
   idade = int(input("Por favor, insira a idade do aluno: "))
except ValueError:
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
        print("Você não inseriu uma idade válida. Por favor,
insira um número inteiro.")
```

Código-fonte 2 – Pedindo a idade do aluno com try/except
Fonte: Elaborado pelo autor (2022)

Nesse exemplo, o usuário é solicitado a inserir a idade do aluno. O valor inserido é convertido para um inteiro com o método int(). No entanto, se o usuário inserir algo que não seja um número, uma exceção ValueError será "lançada" e o controle será passado para o bloco "except". Dentro desse bloco, uma mensagem de erro é exibida, informando ao usuário que ele precisa inserir um número inteiro.

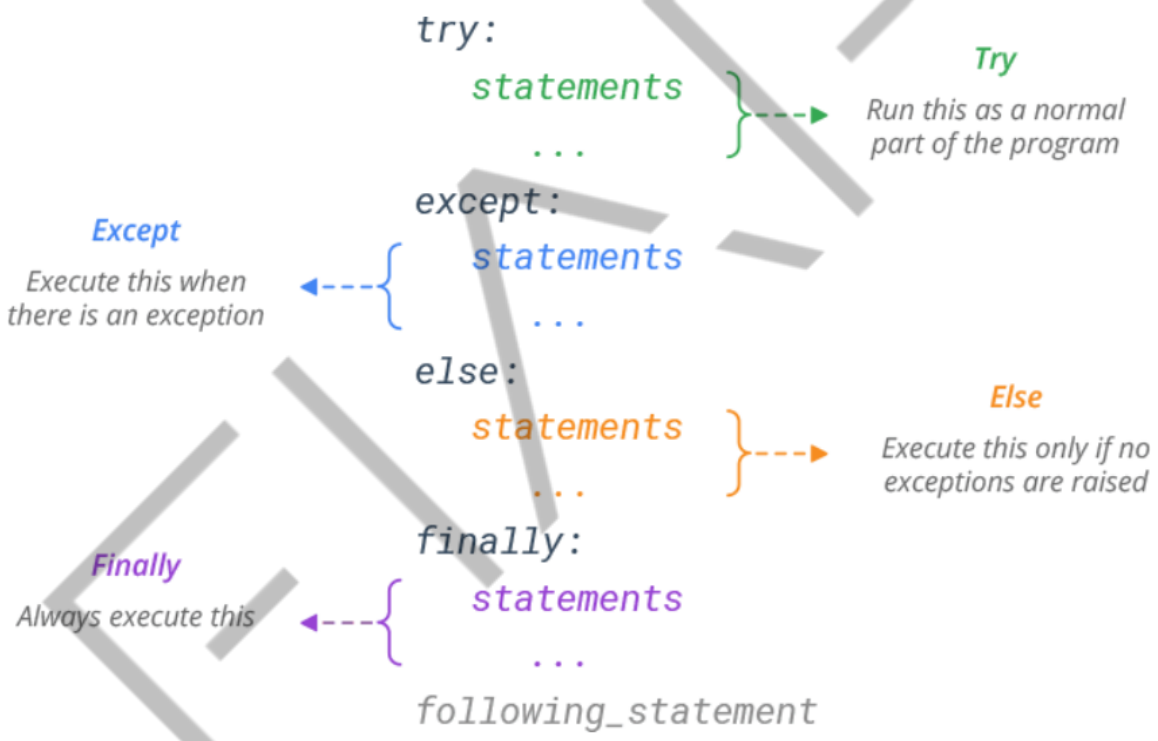


Figura 2 – Esquema geral da exceção
Fonte: Google Imagens (2019)

Os erros inesperados ocorridos em tempo real de execução são geralmente decorrentes de uma falha na codificação do programa fonte, falha de hardware, leitura inconsistente de um determinado conteúdo entre outras origens. Como não é possível antecipar a todos os erros, o planejamento para capturá-los são imprescindíveis em aplicações críticas onde não se permite interrupções.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Imagine agora que você gostaria de exibir a idade do aluno somente se ela for inserida corretamente pelo usuário. Para isso, podemos explorar o bloco "else".

O bloco "else" é usado junto com o bloco "try / except" e é executado se não houver nenhuma exceção lançada dentro do bloco "try". É uma boa prática colocar o código que você espera ser executado sem erros dentro do bloco "try", e colocar o código que depende da execução bem-sucedida do bloco "try" dentro do bloco "else".

```
     try:
        idade = int(input("Por favor, insira a idade do aluno:
"))
     except ValueError:
        print("Você não inseriu uma idade válida. Por favor,
insira um número inteiro.")
     else:
        print("A idade do aluno é:", idade)
```

Código-fonte 3 – Pedindo a idade do aluno com try/except/else
Fonte: Elaborado pelo autor (2022)

Se o usuário inserir uma idade válida (um número inteiro), o bloco "try" será executado sem problemas e o controle será passado para o bloco "else". Dentro desse bloco, uma mensagem é exibida, informando ao usuário a idade do aluno que ele inseriu.

Mas professora, está faltando um bloco ainda: o finally. O que ele faz? O bloco "finally" é usado para garantir que um trecho de código será executado independentemente de uma exceção ser "lançada" ou não. É comum usarmos esse bloco para fechar arquivos ou liberar recursos alocados durante a execução do programa.

```
     try:
        idade = int(input("Por favor, insira a idade do aluno:
"))
     except ValueError:
        print("Você não inseriu uma idade válida. Por favor,
insira um número inteiro.")
     else:
        print("A idade do aluno é:", idade)
     finally:
        print("Obrigado por usar nosso programa!")
```

Código-fonte 4 – Pedindo a idade do aluno com try/except/else/finally
Fonte: Elaborado pelo autor (2022)

Independentemente de uma exceção ser "lançada" ou não, o bloco "finally" será executado e a mensagem "Obrigado por usar nosso programa!" será exibida ao final

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

da execução. Isso garante que, mesmo em caso de erro, algumas ações serão realizadas antes do programa finalizar sua execução.

1.3 O que Python conhece sobre você?

Você já imaginou que o Python pode adivinhar o que você irá fazer de errado durante a execução de um programa? Para isso é que existe o ExceptionType.

ExceptionType é uma informação específica sobre a natureza do erro que ocorreu. É como se o Python dissesse: "Ei, você cometeu um erro porque fez tal coisa". Isso permite que você trate erros específicos de uma maneira específica. Alguns dos tipos de ExceptionType mais comuns são:

- **Exception**: é a classe base para todas as exceções no Python. Se você estiver criando uma nova exceção personalizada, é comum que ela herde desta classe. Ela é uma classe genérica e é aplicável a vários tipos de exceções.
- **ValueError**: é gerado quando uma operação ou função recebe um argumento com um tipo de valor válido, mas com um valor inapropriado. Por exemplo, ao tentar converter uma string para um número inteiro usando a função int() e fornecer uma string que não representa um número.
- **ZeroDivisionError**: é gerado quando você tenta dividir por zero. Por exemplo, tentar calcular 5 / 0 gerará um ZeroDivisionError.
- **TypeError**: é gerado quando você tenta executar uma operação ou função em um objeto de um tipo inapropriado. Por exemplo, ao tentar adicionar uma string e um número.
- **IndexError**: é gerada quando você tenta acessar um índice de uma lista ou outra estrutura de dados que não existe. Por exemplo, tentar acessar o 10º item de uma lista com apenas 9 elementos gerará um IndexError.
- **KeyError**: é gerado quando você tenta acessar uma chave que não existe em um dicionário.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

- **AttributeError**: é gerada quando você tenta acessar um atributo ou método que não existe em um objeto. Por exemplo, tentar acessar um método que não foi definido em uma classe gerará um AttributeError.

Além desses, também há exceções específicas para lidar com arquivos, como:

- **FileNotFoundError**: esta exceção é lançada quando um arquivo não é encontrado ou o caminho para o arquivo é inválido. Por exemplo, se você tentar abrir um arquivo com o caminho especificado e o arquivo não existir, uma FileNotFoundError será lançada.
- **PermissionError**: esta exceção é lançada quando o usuário não tem permissão para acessar ou modificar um arquivo. Por exemplo, se você tentar abrir um arquivo para escrita e o arquivo estiver protegido por permissões de escrita, uma PermissionError será lançada.
- **IsADirectoryError**: esta exceção é lançada quando um arquivo é esperado, mas um diretório é encontrado no caminho especificado. Por exemplo, se você tentar abrir um arquivo, mas o caminho especificado aponta para um diretório, uma IsADirectoryError será lançada.

Talvez você veja algum código por aí com o bloco except sem especificar o tipo um tipo de exceção. Nesse caso, o bloco except capturará qualquer exceção que seja "lançada" durante a execução do bloco try. Isso pode ser útil em situações em que você deseja tratar qualquer tipo de erro de forma genérica.

```
     try:
        idade = int(input("Por favor, insira a idade do aluno: "))
     except:
        print("Ocorreu um erro. Por favor, insira uma idade válida.")
     else:
        print("A idade do aluno é:", idade)
```

Código-fonte 5 – Exemplo com except sem especificar o tipo de exceção
Fonte: Elaborado pelo autor (2022)

Mas cuidado: ao usar o bloco except sem especificar um tipo de exceção, você pode estar "escondendo" erros importantes e tornando mais difícil depurar o seu

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

código. Portanto, é sempre recomendável especificar o tipo de exceção que você deseja tratar.

DICA: Você pode consultar todos os tipos de exceção que o Python trata aqui:

https://docs.python.org/3/library/exceptions.html

1.4 E quando eu quero lançar ou propagar uma exceção?

Em programação, o conceito de escopo se refere ao contexto em que variáveis, funções ou outros objetos são visíveis e acessíveis. Em termos simples, o escopo define o conjunto de partes do programa em que uma variável ou função pode ser acessada ou executada.

Escopos podem ser de dois tipos: global ou local. Variáveis definidas em um escopo global podem ser acessadas em qualquer parte do programa, enquanto variáveis definidas em um escopo local só são acessíveis dentro daquele bloco de código. No contexto de exceções, o escopo é importante porque determina a forma como as exceções são lançadas, capturadas ou ainda propagadas.

Quando uma exceção é lançada, a execução do programa é interrompida e é criada uma pilha de chamadas de métodos, representando a sequência de métodos chamados que levaram à exceção. O processo de captura de exceção consiste em identificar a exceção e fornecer um mecanismo para lidar com ela, como imprimir uma mensagem de erro ou realizar alguma ação para corrigir o erro. Isso é feito com o uso de blocos try / except.

Quando uma exceção é lançada e não é capturada pelo código dentro do escopo local, ela é propaga para o escopo acima, ou seja, o escopo que o chamou. Esse processo continua até que a exceção seja capturada ou até que a pilha de chamadas de métodos tenha sido esgotada, o que resulta em uma exceção não tratada e o programa é interrompido.

Em Python, é possível tratar exceções usando o bloco try / except. No bloco try, colocamos o código que pode causar uma exceção e, no bloco except, colocamos o código que será executado em caso de uma exceção ser lançada. É possível especificar o tipo de exceção a ser capturada, o que permite lidar de forma diferente

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

com diferentes tipos de exceções. Além disso, é possível usar o bloco finally para garantir que algum código seja executado independentemente se uma exceção foi lançada ou não.

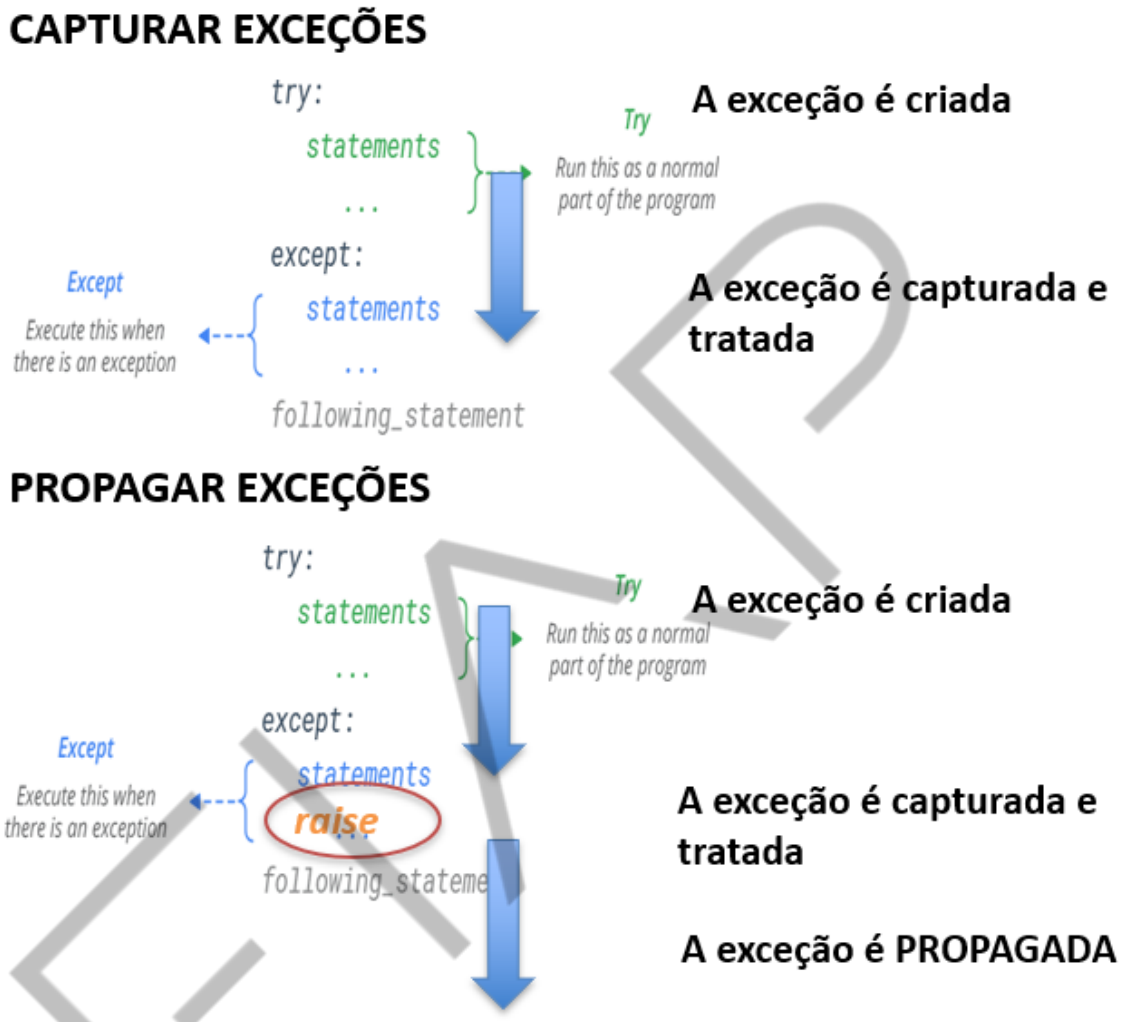


Figura 3 – Esquema lançamento / propagação da exceção
Fonte: Google Imagens adaptado pela autora (2019)

Em Python, o lançamento e a propagação de exceções são feitas por meio do comando raise porque ele é uma forma fácil e direta de forçar a ocorrência de uma exceção, permitindo que o programador controle o fluxo do programa em caso de erros. Quando ocorre uma exceção no seu código, o interpretador Python interrompe a execução normal do programa e procura um bloco de código except que corresponda à exceção. O comando raise permite que você lance uma exceção específica e direcione o fluxo do programa para o primeiro bloco except que possa lidar com essa exceção.

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

O uso do raise também permite que você personalize a mensagem de erro, fornecendo mais informações sobre o problema que ocorreu. Por exemplo, ao invés de simplesmente retornar um erro genérico, você pode fornecer uma mensagem detalhada explicando o que está acontecendo e ajudando a solucionar o problema de forma mais eficiente.

```
    try:
        idade = int(input("Insira sua idade: "))
        if idade < 0:
            raise ValueError("Idade não pode ser menor que
0")
    except ValueError as error:
        print("Erro:", error)
```

Código-fonte 6 – Exemplo de raise com especificação da mensagem de erro
Fonte: Elaborado pelo autor (2022)

Nesse exemplo, estamos usando o raise dentro do bloco if para verificar se a idade inserida é negativa. Se for, uma exceção ValueError é lançada com uma mensagem personalizada "Idade não pode ser menor que 0". Em seguida, usamos o except para capturar a exceção e exibir a mensagem de erro.

Além disso, o raise também permite que você processe uma exceção e, em seguida, a lance novamente para ser tratada em outro lugar no código. Essa capacidade de propagação de exceções é crucial para a manutenção da integridade dos dados e para garantir que o programa sempre saia de uma situação de erro de maneira controlada.

```
    try:
        number = int(input("Insira um número: "))
        if number < 0:
            raise ValueError("Número não pode ser negativo")
    except ValueError as error:
        print("Erro:", error)
        raise
```

Código-fonte 7 – Exemplo de raise com especificação da mensagem de erro e propagação
Fonte: Elaborado pelo autor (2022)

Nesse exemplo, estamos lançando uma exceção ValueError se o número inserido for negativo. O bloco except é usado para capturar a exceção e exibir a mensagem de erro. Em seguida, usamos o raise sem argumentos para lançar novamente a exceção, permitindo que seja tratada em outro local ou mesmo nesse caso, interrompa a execução do programa

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

1.5 Exceções Personalizadas

Imagine, no nosso exemplo de idade, que você quer personalizar uma exceção para que a idade fornecida pelo usuário não seja superior a 125 anos, afinal, a francesa que mais viveu, Jeanne Calment, tinha 122 anos quando faleceu. Essa não é uma situação de erro comum que o Python está acostumado a tratar. Para isso podemos criar uma exceção personalizada completa, a partir de uma classe base.

A classe base é a classe a partir da qual uma nova classe é derivada. Em Python, a classe Exception é a classe base para todas as exceções.

1.5.1 Classe base Exception

A classe base de exceção no Python é a classe "Exception". Essa classe tem vários métodos que podem ser sobrepostos ou utilizados em suas subclasses personalizadas de exceção. Aqui estão alguns dos métodos mais comuns:

- **init**: é o construtor da classe, que é executado quando uma nova instância da classe é criada. É usado para inicializar as propriedades da classe.
- **str**: é o método que é chamado quando o objeto da classe é convertido em uma string. Por padrão, esse método retorna a string padrão "Exception". Mas pode ser sobrescrito para personalizar a mensagem de erro.
- **repr**: é o método que é chamado quando o objeto da classe é representado como uma string. Por padrão, esse método retorna a string padrão "Exception". Mas pode ser sobrescrito para personalizar a mensagem de erro.
- **with_traceback:** é o método que permite adicionar um rastreamento a uma exceção. É usado para anexar informações adicionais a uma exceção, como o local onde ocorreu o erro.
- **args**: é uma tupla de argumentos que podem ser passados para o construtor da classe.

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

Esses são apenas alguns dos métodos mais comuns da classe "Exception". No entanto, há outros métodos disponíveis que podem ser usados para personalizar exceções de acordo com as necessidades do seu programa.

Quando queremos personalizar a mensagem de erro de uma exceção, é comum sobrepor o método str na nova classe que herda da classe Exception. Dessa forma, é possível fornecer uma mensagem mais informativa e específica para o erro que está sendo gerado.

Voltando ao nosso exemplo, ao criarmos uma classe de exceção personalizada para lidar com idades superiores a 125 anos, estamos sendo mais específicos e precisos na maneira como tratamos esse tipo de erro. Ao lançarmos essa exceção, podemos fornecer uma mensagem específica e relevante para esse caso, facilitando a compreensão e solução do erro pelo programador.

```
class IdadeMaximaExcedida(Exception):
    def __str__(self):
        return "A idade não pode ser superior a 125 anos."
try:
    idade = int(input("Insira sua idade: "))
    if idade < 0:
        raise ValueError("Idade não pode ser menor que
0")
    elif idade > 125:
        raise IdadeMaximaExcedida
except ValueError as error:
    print("Erro:", error)
except IdadeMaximaExcedida as error:
    print("Erro:", error)
else:
  print(f"Você tem {idade} anos!")
```

Código-fonte 8 – Exemplo de raise com especificação da mensagem de erro e propagação
Fonte: Elaborado pelo autor (2022)

Agora, caso o usuário informe uma idade maior que 125 anos, a exceção IdadeMaximaExcedida será lançada e será exibida uma mensagem de erro personalizada.

DICA: Para capturar a mensagem de erro quando estamos tratando a exceção genericamente podemos usar a classe base Exception e usando um apelido “error” conseguimos recuperar a mensagem de erro

except Exception as error:

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
    print("Erro generico: ", error)
```

2 HANDS ON: HORA DA PRÁTICA

2.1 Com ou sem exceção

É fundamental que conheçamos as exceções para entender como tratar situações inesperadas durante a execução de um programa.

Quando um erro ocorre, podemos capturar a exceção e tomar uma ação específica, o que ajuda a evitar o encerramento abrupto do programa e garante que ele continue a executar da melhor forma possível.

Você pode salvar a vida de um usuário personalizando as mensagens de erro para torná-las mais úteis e fáceis de compreender. Vamos praticar

2.1.1 Quanto estamos gastando mesmo?

Por incrível que pareça vamos ao supermercado toda hora; tanto que quando percebemos estamos sem uma ideia de quanto estamos gastando.

Para ajudar podemos fazer um programa que nos ajude a calcular a média de quanto gastamos por item comprado.

Mas temos que nos certificar de que tudo ocorra corretamente: nada de deixar, por exemplo, que entremos com uma letra aonde está sendo pedido algum valor. Também vamos garantir que em nosso cálculo não haja divisão por zero. Quem vai nos ajudar a tratar esse tipo de comportamento é a exceção.

Será que conseguimos tratar dois tipos diferentes de exceção num mesmo trecho de código?

```
     titulo = 'Valor Medio do Item Comprado'
     print(f'{titulo:50}')
     try:
       vltotal = float(input('Qual o valor total da compra:
R$'))
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
        qtitens = int(input('Quantos itens na compra: '))
        media = round(vltotal / qtitens)
      except ZeroDivisionError:
        print('Divisão por zero! O valor para Item deve ser
maior que 0 !')
      except ValueError:
        print('Entre com um número!')
      else:
        print(f'Valor total: R$ {vltotal:.2f}, Quantidade de
Itens: {qtitens}, Média por Item: R$ {media:.2f}')
      finally:
        print('Continue acompanhando seus gastos!')
```

Código-fonte 9 – Média de valor gasto por item em uma compra
Fonte: Elaborado pelo autor (2022)

Notou que separamos cada um dos tipos de exceções em blocos diferentes? E se quiséssemos acrescentar o tratamento de uma exceção genérica para outros erros não previstos? Será que poderíamos inserir um outro bloco de exceção?

A resposta é sim! Nesse caso podemos acrescentar um bloco except antes do bloco else, emitindo uma mensagem de erro geral. A sugestão é que você faça essa alteração e faça os testes.

2.1.2 Nada de ficar devendo!

Uma evolução do programa anterior é impedir que os valores gastos e a quantidade de itens comprados sejam negativos. Como isso não é uma situação muito comum podemos criar uma classe de exceção personalizada e usar nessa situação. Dá uma olhada em como seria:

```
      titulo = 'Valor Medio do Item Comprado'
      print(f'{titulo:50}')
      class NumerosNegativos (Exception):
          def __str__(self):
               return "Você forneceu um número negativo"
      try:
        vltotal = float(input('Qual o valor total da compra:
R$'))
        qtitens = int(input('Quantos itens na compra: '))
        if vltotal < 0 or qtitens < 0:
          raise NumerosNegativos
        media = round(vltotal / qtitens)
      except ZeroDivisionError:
        print('Divisão por zero! O valor para Item deve ser
maior que 0 !')
```

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

```
      except ValueError:
        print('Entre com um número!')
      except NumerosNegativos as error:
        print(error)
      else:
        print(f'Valor total: R$ {vltotal:.2f}, Quantidade de
Itens: {qtitens}, Média por Item: R$ {media:.2f}')
      finally:
        print('Continue acompanhando seus gastos!')
```

Código-fonte 10 – Média de valor gasto por item em uma compra com erro personalizado
Fonte: Elaborado pelo autor (2022)

A classe aqui nos dá possibilidade de fazer outros tratamentos, mas nós queremos apenas enviar uma mensagem amigável ao usuário, e por isso não sobreporemos qualquer outro método.

Com tanto aprendizado, a partir de agora você só abre exceção se quiser!

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com

REFERÊNCIAS

PIVA, Dilermando Júnior. **Algoritmos e programação de computadores**. São Paulo: Elsevier, 2012.

PUGA, Sandra; RISSETTI, Gerson. **Lógica de programação e estrutura de dados**. São Paulo: Pearson Prentice Hall, 2009.

RAMALHO, Luciano. **Python Fluente**: Programação clara, concisa e eficiente. São Paulo: Novatec, 2015.

PYTHON; Et al. Documentação oficial do Python. 2023. Disponível em: < https://docs.python.org/3/>. Acesso em: 04 fev 2023

FIAP

PDF exclusivo para Yago Campos Silva - rm560063
yago.campos.s@hotmail.com